

Table of contents

Recherche Tabou (Tabu Search)	1
Introduction	1
Principes de base	1
Pourquoi la liste taboue ne peut-elle pas être permanente ?	2
Convergence de la TS	2
La liste taboue (ou liste d'exclusion)	2
Objectifs	2
Construction de la liste taboue	2
Gestion de la liste taboue	3
Exploration vs exploitation	4
Application au problème d'affectation quadratique (QAP)	4
Définition du QAP	4
Exemple de résolution avec la recherche tabou	4
Synthèse de l'exemple	5
Conclusion	5

Recherche Tabou (Tabu Search)

Introduction

La **Recherche Tabou** (Tabu Search, TS) est une métaheuristique introduite par Fred Glover en 1986. Elle est conçue pour guider les recherches locales au-delà des optima locaux en évitant de revenir à des solutions déjà visitées.

Elle s'appuie sur des méthodes de recherche locale (comme la montée de colline) et utilise une structure de mémoire pour :

- Échapper aux optima locaux
- Éviter de revisiter des solutions récemment explorées

Ainsi, elle explore l'espace de recherche de manière plus large et "intelligente".

Principes de base

La TS est bien adaptée aux problèmes d'optimisation combinatoire, comme le problème d'affectation quadratique (QAP). Comme la plupart des métaheuristicues, ses principes fondamentaux sont assez simples.

L'exploration se fait de voisin en voisin :

$$x_n \rightarrow x_{n+1} \in V(x_n)$$

L'opération d'exploration cherche le "meilleur" voisin $x_{n+1} \in V(x_n)$, où "meilleur" signifie :

- Une solution non taboue
- Une fitness optimale dans $V(x_n) \setminus \{x_n\}$ (peu importe la valeur de $f(x_n)$, la fitness peut se dégrader)

- Si plusieurs points ont la même fitness optimale, on en sélectionne un au hasard

La spécificité clé de la TS est l'existence de points ou de mouvements interdits (tabous). L'objectif de cette liste taboue est d'empêcher l'opérateur de recherche de revenir à des configurations déjà explorées.

L'attribut tabou n'est pas permanent : les points/mouvements interdits sont stockés dans une mémoire à court terme, et la liste taboue est mise à jour constamment pendant le processus d'exploration.

Pourquoi la liste taboue ne peut-elle pas être permanente ?

Si la liste taboue était permanente, elle pourrait bloquer le processus de recherche. Par exemple, dans un espace discret avec un voisinage de type NEWS (Nord, Est, Sud, Ouest), une liste permanente empêcherait tout mouvement après un certain temps, car tous les mouvements deviendraient interdits.

Convergence de la TS

La convergence de la TS dépend de plusieurs facteurs :

- La longueur de la mémoire (taille de la liste taboue)
- La taille et la structure du voisinage
- La condition initiale
- Le critère d'arrêt
- Le paysage de fitness

Un résultat mathématique assure que, sous certaines conditions (espace de recherche fini, voisinage symétrique, accessibilité en un nombre fini d'étapes), une TS qui stocke tous les points visités et peut revisiter le plus ancien point de la liste explorera tout l'espace de recherche et trouvera l'optimum global avec certitude.

Cependant, ce résultat a peu de pertinence pratique car il ne donne pas d'indication sur le temps nécessaire (qui peut être exponentiel) et requiert une mémoire arbitrairement longue.

La liste taboue (ou liste d'exclusion)

Objectifs

La liste taboue a pour buts :

- D'éviter de ré-échantillonner des configurations déjà visitées
- D'améliorer l'efficacité du processus d'exploration de l'espace de recherche S

Construction de la liste taboue

On peut inclure dans la liste taboue :

- Les configurations précédemment visitées
- Les attributs de ces configurations (comme la fitness)
- Souvent, les mouvements interdits m_j parmi ceux définissant le voisinage

Le voisinage est défini par un ensemble de mouvements :

$$V(s) = \{s' \in S \mid s' = m_i(s), i = 1, \dots, \ell\}$$

On peut aussi interdire les mouvements inverses de ceux déjà effectués. Par exemple, pour un marcheur en 2D avec un voisinage NEWS :

- Après un mouvement Nord, on interdit le mouvement Sud
- Après un mouvement Est, on interdit le mouvement Ouest

Ces stratégies évitent les cycles simples qui pourraient bloquer la recherche.

Gestion de la liste taboue

La liste taboue ne peut être permanente : des mécanismes sont nécessaires pour supprimer les éléments tabous. On distingue deux types de mémoire :

- Mémoire à court terme
- Mémoire à long terme

Mémoire à court terme

La mémoire à court terme a une taille limitée M . Elle peut être gérée de deux façons :

1. Utiliser une liste taboue de taille finie et ne conserver que les exclusions les plus récentes (FIFO : premier entré, premier sorti)
2. Associer aux données de la liste taboue une durée d'exclusion (temps de bannissement k)

L'approche la plus courante est la seconde : chaque mouvement tabou est conservé pendant k itérations. k est appelé le *temps de bannissement* ou *tabu tenure*. La valeur optimale de k n'est pas connue et est généralement choisie aléatoirement selon une distribution de probabilité dont les paramètres sont déterminés empiriquement.

Mémoire à long terme

L'utilisation répétée du même mouvement peut être acceptable à court terme, mais pas à long terme. Pour éviter des biais dans la sélection des mouvements, on peut :

- Collecter des informations sur toute la trajectoire d'exploration depuis le début
- Générer des statistiques sur les mouvements/attributs/solutions passés
- Pénaliser les mouvements trop fréquents pour favoriser les moins fréquents (promouvoir l'exploration)

Ce système de bonus-malus peut être modulé par la valeur de la fitness pour équilibrer exploration et exploitation.

Utilisation combinée des mémoires

On peut combiner et/ou alterner l'utilisation des mémoires à court et à long terme. Il faut introduire des paramètres de contrôle pour les deux types de mémoire, qui s'ajoutent aux autres paramètres de la métaheuristique (comme M et k). Les valeurs optimales de ces paramètres ne sont pas connues.

Exploration vs exploitation

Le nombre d'entrées taboues ou de mouvements tabous influence le processus d'exploration :

- Plus le nombre de mouvements interdits est grand, plus on choisit des voisins avec une fitness plus faible : c'est l'**exploration** ou **diversification**
- Si ce nombre est petit, on est libre d'utiliser des solutions à haute fitness : c'est l'**exploitation** ou **intensification**

Un autre ingrédient peut être intégré : l'**aspiration**. Si un point dans le voisinage a une fitness supérieure à celles rencontrées auparavant, ce point est sélectionné, même si le mouvement est interdit (tabou contourné).

Application au problème d'affectation quadratique (QAP)

Définition du QAP

Le QAP est un problème d'optimisation combinatoire important et difficile. On dispose de :

- n objets et n emplacements possibles pour ces objets
- Des distances d_{rs} entre les emplacements r et s
- Des poids ou flux f_{ij} entre les paires d'objets (i, j)

Le but est de trouver le placement optimal (une permutation) des n objets sur les n emplacements pour minimiser la fitness :

$$f = \sum_{i,j} f_{ij} d_{r_i, r_j}$$

où r_i est la position de l'objet i . L'espace de recherche S est l'ensemble des permutations de n objets.

Le QAP apparaît dans de nombreux domaines : agencement d'installations, conception électronique, analyse de données, conception d'hôpitaux, ordonnancement, etc.

Exemple de résolution avec la recherche tabou

Pour résoudre le QAP avec la TS, on définit :

- L'espace de recherche : toutes les permutations
- Le voisinage : on peut échanger deux objets contigus, échanger deux objets quelconques, ou déplacer et insérer un objet

L'évaluation du voisinage peut être faite efficacement en calculant la différence de fitness $\Delta_i = f(m_i(\mathbf{p})) - f(\mathbf{p})$ pour chaque mouvement m_i . Pour une minimisation, on cherche $\Delta_i < 0$.

La liste taboue peut être implémentée sous forme d'une matrice $n \times n$ T , où l'entrée T_{ir} correspond à l'itération à laquelle l'objet i a quitté l'emplacement r , à laquelle on ajoute le temps de bannissement k . Un mouvement (r, s) est interdit si $T_{i_s r}$ et $T_{i_r s}$ contiennent toutes deux une valeur supérieure au compteur d'itérations courant.

Le temps de bannissement k est généralement tiré aléatoirement dans un intervalle, par exemple $k \in [0.9 \times n, 1.1 \times n + 4]$.

Un exemple classique est le problème benchmark NUG5 ($n = 5$). La TS peut trouver l'optimum global en acceptant parfois une dégradation de la fitness pour explorer de nouvelles régions de l'espace de recherche.

Synthèse de l'exemple

L'idée derrière l'exemple est de montrer comment la TS, en utilisant une liste taboue adaptative, peut éviter les cycles et échapper aux optima locaux. En acceptant temporairement des solutions moins bonnes, l'algorithme peut explorer davantage l'espace et finalement converger vers une solution de haute qualité.

Conclusion

La recherche tabou est une métaheuristique puissante pour l'optimisation combinatoire. Son principe de base est simple : utiliser une mémoire (liste taboue) pour éviter de revisiter des solutions et ainsi explorer l'espace de recherche plus efficacement. La gestion de la liste taboue (mémoire à court et long terme) et l'équilibre entre exploration et exploitation sont des aspects clés de la méthode.

Son application à des problèmes comme le QAP montre son efficacité pratique. Cependant, comme pour beaucoup de métaheuristiques, le réglage des paramètres (taille de la liste, temps de bannissement, etc.) reste un défi et est souvent empirique.

La TS est un outil important dans la boîte à outils de l'optimisation, en particulier pour les problèmes difficiles où les méthodes exactes ne sont pas réalisables.